

```
In [1]: import json
import random
import numpy as np
import pandas as pd
import torch

from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup
)
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

```
/home/jay/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
In [2]: import sys
import torch

print("Python path:", sys.executable)
print("Torch path:", torch.__file__)
print("Torch version:", torch.__version__)
print("CUDA Available:", torch.cuda.is_available())
```

```
Python path: /usr/bin/python3
Torch path: /home/jay/.local/lib/python3.10/site-packages/torch/__init__.py
Torch version: 2.5.1+cu121
CUDA Available: True
```

```
In [3]: """
Configuration
"""

MODEL_NAME = "xlm-roberta-base"

DATASET_PATH = "hf://datasets/mohanrj/MYBully/"
splits = {"train": "train.csv", "validation": "val.csv"}
TRAIN_DATA_PATH = DATASET_PATH + splits["train"]
VALID_DATA_PATH = DATASET_PATH + splits["validation"]

TEXT_COLUMN = "Tweet"
LABEL_COLUMN = "Emotion"

MAX_LENGTH = 140
BATCH_SIZE = 64
EPOCHS = 5
LEARNING_RATE = 2e-5
FOCAL_GAMMA = 1.0

RANDOM_SEED = 42
```

```

NUM_GPUS = min(2, torch.cuda.device_count())
DEVICE = torch.device("cuda:0" if NUM_GPUS else "cpu")

print("Using device:", DEVICE)
print("GPUs used:", NUM_GPUS)

```

Using device: cuda:0
GPUs used: 2

```

In [4]: """
        Reproducibility
        """

        def set_seed(seed):
            random.seed(seed)
            np.random.seed(seed)
            torch.manual_seed(seed)

            if torch.cuda.is_available():
                torch.cuda.manual_seed_all(seed)

        set_seed(RANDOM_SEED)

```

```

In [5]: """
        Load datasets
        """

        train_df = pd.read_csv(TRAIN_DATA_PATH, usecols=[TEXT_COLUMN, LABEL_COLUMN])

        # Keep only rows with text and emotion labels.
        train_df = train_df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN]).copy()
        train_df[LABEL_COLUMN] = train_df[LABEL_COLUMN].astype(str).str.strip()

        print("Number of examples:", len(train_df))
        print("\nClass distribution:")
        print(train_df[LABEL_COLUMN].value_counts())

```

Number of examples: 3510

Class distribution:

```

Emotion
Anger      1830
Neutral    731
Happiness  324
Disgust    310
Sadness    207
Fear       61
Surprise   47
Name: count, dtype: int64

```

```

In [6]: """
        Split validation data into validation and test sets
        """

        source_valid_df = pd.read_csv(VALID_DATA_PATH, usecols=[TEXT_COLUMN, LABEL_COLUMN])
        source_valid_df = source_valid_df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN]).copy()
        source_valid_df[LABEL_COLUMN] = source_valid_df[LABEL_COLUMN].astype(str).str.strip()

```

```
valid_df, test_df = source_valid_df, source_valid_df.copy()

print("Number of test examples:", len(test_df))
print("\nTest class distribution:")
print(test_df[LABEL_COLUMN].value_counts())
```

Number of test examples: 1171

Test class distribution:

```
Emotion
Anger      620
Neutral    240
Happiness  113
Disgust     83
Sadness     69
Surprise    27
Fear        19
Name: count, dtype: int64
```

```
In [7]: print("Number of validation examples:", len(valid_df))
print("\nValidation class distribution:")
print(valid_df[LABEL_COLUMN].value_counts())
```

Number of validation examples: 1171

Validation class distribution:

```
Emotion
Anger      620
Neutral    240
Happiness  113
Disgust     83
Sadness     69
Surprise    27
Fear        19
Name: count, dtype: int64
```

```
In [8]: """
Encode labels
"""

labels = sorted(train_df[LABEL_COLUMN].unique())
label2id = {label: index for index, label in enumerate(labels)}
id2label = {index: label for label, index in label2id.items()}
NUM_LABELS = len(labels)

for df in [train_df, valid_df, test_df]:
    unknown_labels = set(df[LABEL_COLUMN]) - set(label2id)
    if unknown_labels:
        raise ValueError(f"Labels in data split not found in training set: {sort
df["label_id"] = df[LABEL_COLUMN].map(label2id)

print("\nEmotion labels:")
print(label2id)
```

Emotion labels:

```
{'Anger': 0, 'Disgust': 1, 'Fear': 2, 'Happiness': 3, 'Neutral': 4, 'Sadness': 5,
'Surprise': 6}
```

```
In [9]: """
Train / validation / test split
"""
```

```

print("\nDataset sizes:")
print("Train:", len(train_df))
print("Validation:", len(valid_df))
print("Test:", len(test_df))

```

Dataset sizes:
Train: 3510
Validation: 1171
Test: 1171

```

In [10]: """
          Tokenizer
          """

          tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, timeout=120)

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```

In [11]: """
          PyTorch Dataset
          """

          class EmotionDataset(Dataset):

              def __init__(self, dataframe, tokenizer, max_length):
                  self.texts = dataframe[TEXT_COLUMN].tolist()
                  self.labels = dataframe["label_id"].astype(int).tolist()

                  self.tokenizer = tokenizer
                  self.max_length = max_length

              def __len__(self):
                  return len(self.texts)

              def __getitem__(self, index):

                  text = str(self.texts[index])
                  label = self.labels[index]

                  encoding = self.tokenizer(
                      text,
                      truncation=True,
                      padding="max_length",
                      max_length=self.max_length,
                      return_tensors="pt"
                  )

                  return {
                      "input_ids": encoding["input_ids"].squeeze(0),
                      "attention_mask": encoding["attention_mask"].squeeze(0),
                      "labels": torch.tensor(label, dtype=torch.long)
                  }

          train_dataset = EmotionDataset(train_df, tokenizer, MAX_LENGTH)
          val_dataset = EmotionDataset(valid_df, tokenizer, MAX_LENGTH)
          test_dataset = EmotionDataset(test_df, tokenizer, MAX_LENGTH)

```

```
In [12]: """
DataLoaders
"""

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

```
In [13]: """
Initiating model
"""

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
    id2label=id2label,
    label2id=label2id,
    problem_type="single_label_classification"
)

model.to(DEVICE)

if NUM_GPUS > 1:
    model = torch.nn.DataParallel(
        model,
        device_ids=list(range(NUM_GPUS)),
        output_device=0
    )
    print(f"Using DataParallel across {NUM_GPUS} GPUs")
```

```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 15028.70it/s]
[transformers] XLMRobertaForSequenceClassification LOAD REPORT from: xlm-roberta-
base
```

Key	Status
roberta.pooler.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
roberta.pooler.dense.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING
classifier.out_proj.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Using DataParallel across 2 GPUs

```
In [14]: """
Optimizer and scheduler
"""

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE
)

total_training_steps = (
    len(train_loader) * EPOCHS
)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_training_steps),
    num_training_steps=total_training_steps
)
```

```
In [15]: """
Focal loss
"""

class_counts = torch.bincount(
    torch.tensor(train_dataset.labels),
    minlength=NUM_LABELS
).to(DEVICE)
class_weights = class_counts.sum() / (
    NUM_LABELS * class_counts.clamp(min=1)
)

def focal_loss(logits, targets):
    cross_entropy = torch.nn.functional.cross_entropy(
        logits,
```

```

        targets,
        reduction="none"
    )
    probability_of_target = torch.exp(-cross_entropy)
    target_weights = class_weights[targets]

    return (
        target_weights
        * (1 - probability_of_target).pow(FOCAL_GAMMA)
        * cross_entropy
    ).mean()

```

```

In [16]: """
Evaluation function
"""

def evaluate(model, dataloader):

    model.eval()

    all_predictions = []
    all_labels = []

    total_loss = 0

    with torch.no_grad():

        for batch in dataloader:

            input_ids = batch["input_ids"].to(DEVICE)
            attention_mask = batch["attention_mask"].to(DEVICE)
            labels_batch = batch["labels"].to(DEVICE)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask
            )
            loss = focal_loss(outputs.logits, labels_batch)

            total_loss += loss.item()
            predictions = outputs.logits.argmax(dim=-1)

            all_predictions.extend(predictions.cpu().tolist())
            all_labels.extend(labels_batch.cpu().tolist())

    average_loss = total_loss / len(dataloader)

    accuracy = accuracy_score(
        all_labels,
        all_predictions
    )

    macro_f1 = f1_score(
        all_labels,
        all_predictions,
        average="macro",
        zero_division=0
    )

    weighted_f1 = f1_score(

```

```

        all_labels,
        all_predictions,
        average="weighted",
        zero_division=0
    )

    return {
        "loss": average_loss,
        "accuracy": accuracy,
        "macro_f1": macro_f1,
        "weighted_f1": weighted_f1,
        "labels": all_labels,
        "predictions": all_predictions
    }

```

```

In [17]: """
Train with focal loss
"""

best_model_path = "best_mybully_focal_model.pt"
best_validation_macro_f1 = -1.0

for epoch in range(EPOCHS):

    model.train()
    total_training_loss = 0

    for batch in train_loader:

        input_ids = batch["input_ids"].to(DEVICE)
        attention_mask = batch["attention_mask"].to(DEVICE)
        labels_batch = batch["labels"].to(DEVICE)

        optimizer.zero_grad(set_to_none=True)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        loss = focal_loss(outputs.logits, labels_batch)
        loss.backward()

        optimizer.step()
        scheduler.step()
        total_training_loss += loss.item()

    average_training_loss = total_training_loss / len(train_loader)
    validation_results = evaluate(model, val_loader)

    print(
        f"Epoch {epoch + 1}/{EPOCHS} | "
        f"Train focal loss: {average_training_loss:.4f} | "
        f"Validation focal loss: {validation_results['loss']:.4f} | "
        f"Validation macro-F1: {validation_results['macro_f1']:.4f}"
    )

    if validation_results["macro_f1"] > best_validation_macro_f1:
        best_validation_macro_f1 = validation_results["macro_f1"]

```

```
checkpoint_model = model.module if isinstance(model, torch.nn.DataParallel)
torch.save(checkpoint_model.state_dict(), best_model_path)
```

```
Epoch 1/5 | Train focal loss: 1.6853 | Validation focal loss: 1.7979 | Validation
macro-F1: 0.0835
Epoch 2/5 | Train focal loss: 1.6334 | Validation focal loss: 1.6711 | Validation
macro-F1: 0.1853
Epoch 3/5 | Train focal loss: 1.4456 | Validation focal loss: 1.4408 | Validation
macro-F1: 0.3229
Epoch 4/5 | Train focal loss: 1.2413 | Validation focal loss: 1.4246 | Validation
macro-F1: 0.3555
Epoch 5/5 | Train focal loss: 1.0812 | Validation focal loss: 1.4231 | Validation
macro-F1: 0.3526
```

```
In [18]: """
Load best model
"""

model_state = torch.load(
    best_model_path,
    map_location=DEVICE
)

if isinstance(model, torch.nn.DataParallel):
    model.module.load_state_dict(model_state)
else:
    model.load_state_dict(model_state)
```

```
/tmp/ipykernel_1479029/1745357026.py:5: FutureWarning: You are using `torch.load`
with `weights_only=False` (the current default value), which uses the default pic
kle module implicitly. It is possible to construct malicious pickle data which wi
ll execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release,
the default value for `weights_only` will be flipped to `True`. This limits the f
unctions that could be executed during unpickling. Arbitrary objects will no long
er be allowed to be loaded via this mode unless they are explicitly allowlisted b
y the user via `torch.serialization.add_safe_globals`. We recommend you start set
ting `weights_only=True` for any use case where you don't have full control of th
e loaded file. Please open an issue on GitHub for any issues related to this expe
rimental feature.
    model_state = torch.load(
```

```
In [19]: """
Final test evaluation
"""

test_results = evaluate(
    model,
    test_loader
)

print("\n=====")
print("FINAL MYBULLY TEST RESULTS")
print("=====")

print(
    f"Accuracy:      "
    f"{test_results['accuracy']:.4f}"
)
```

```

print(
    f"Macro-F1:      "
    f"{test_results['macro_f1']:.4f}"
)

print(
    f"Weighted-F1:   "
    f"{test_results['weighted_f1']:.4f}"
)

```

```

=====
FINAL MYBULLY TEST RESULTS
=====
Accuracy:      0.5209
Macro-F1:      0.3555
Weighted-F1:   0.5566

```

```

In [20]: """
Per-emotion results
"""

print("\nClassification Report:")

print(
    classification_report(
        test_results["labels"],
        test_results["predictions"],
        labels=list(range(NUM_LABELS)),
        target_names=labels,
        digits=4,
        zero_division=0
    )
)

```

```

Classification Report:

```

	precision	recall	f1-score	support
Anger	0.7939	0.6339	0.7049	620
Disgust	0.2081	0.3735	0.2672	83
Fear	0.0625	0.2105	0.0964	19
Happiness	0.4904	0.4513	0.4700	113
Neutral	0.6429	0.3375	0.4426	240
Sadness	0.3103	0.6522	0.4206	69
Surprise	0.0568	0.1852	0.0870	27
accuracy			0.5209	1171
macro avg	0.3664	0.4063	0.3555	1171
weighted avg	0.6348	0.5209	0.5566	1171

```

In [21]: """
Confusion matrix
"""

category_confusion_matrix = confusion_matrix(
    test_results["labels"],
    test_results["predictions"],
    labels=list(range(NUM_LABELS))
)

print("\nEmotion Confusion Matrix:")

```

```
print(  
    pd.DataFrame(  
        category_confusion_matrix,  
        index=labels,  
        columns=labels  
    )  
)
```

Emotion Confusion Matrix:

	Anger	Disgust	Fear	Happiness	Neutral	Sadness	Surprise
Anger	393	71	29	17	32	43	35
Disgust	29	31	4	5	4	7	3
Fear	3	5	4	0	0	6	1
Happiness	7	8	3	51	8	14	22
Neutral	51	27	16	23	81	23	19
Sadness	9	3	7	1	1	45	3
Surprise	3	4	1	7	0	7	5